1. Mục tiêu

- Kế thừa và mở rộng trực tiếp từ Buổi 10 (Jenkins CI/CD): thay vì chỉ có 1 pipeline/1 môi trường, buổi này xây dựng đủ **3 môi trường Dev – Staging – Production**, mỗi môi trường có pipeline, cấu hình và máy chạy riêng.
- Vẫn giữ đầy đủ yêu cầu Unit Test như Buổi 10 (test bắt buộc phải chặn được pipeline khi fail) — buổi này không thay thế mà bổ sung thêm vào quy trình đã có.
- Biết chia code theo **nhánh Git** tương ứng với từng môi trường, và cấu hình Jenkins Multibranch Pipeline để build đúng luồng theo từng nhánh.
- Hiểu và áp dụng đúng nguyên tắc: **Env/Config/Secret của Staging và Production không được nằm trong source code/repository**, mà phải có sẵn từ trước ngay tại máy đang build/deploy môi trường đó.
- Minh họa rõ sự tách biệt Secret giữa các môi trường bằng một tính năng đăng nhập dùng JWT: mỗi môi trường có secret key ký token khác nhau.
- Biết cách cấu hình Jenkins để **rollback** — quay lại một phiên bản code/image trước đó khi phát hiện lỗi ở Staging/Production, mà không cần build lại.
- Mô phỏng nhiều server thật bằng cách coi máy của từng thành viên trong nhóm là một server riêng biệt (Dev/Staging/Production), giao tiếp với nhau qua cùng một mạng LAN.
- Chuẩn bị tốt cho phần thuyết trình và demo trên nhiều máy, thể hiện được toàn bộ vòng đời: code mới → CI theo nhánh → deploy đúng môi trường → rollback khi cần.

2. Nội dung cần nghiên cứu và chuẩn bị

Buổi này **không làm lại từ đầu** mà lắp thêm 4 mảnh còn thiếu vào pipeline đã có ở Buổi 10: (1) chia môi trường theo nhánh Git, (2) Env/Config/Secret tách biệt và không nằm trong code, (3) khả năng rollback, (4) mô phỏng nhiều server bằng máy các thành viên trong nhóm.

2.1. Ôn lại nền tảng từ Buổi 10 và những gì cần bổ sung

- Buổi 10 đã có: Private repo, Jenkins, Jenkinsfile với stage Checkout → Test → Build Image (gắn tag) → Push Image lên registry → Trigger Deploy qua HTTP webhook sang 1 máy CD.
- Buổi 11 bổ sung: pipeline chạy đúng theo **từng nhánh** ứng với từng môi trường, mỗi môi trường đọc đúng file cấu hình/secret riêng của nó, và có khả năng **quay lại phiên bản cũ** khi môi trường đang chạy gặp lỗi.

2.2. Chia môi trường theo nhánh Git (Branching Strategy)

- **Chiến lược nhánh đơn giản, phù hợp bài học:**
  - `dev`: Nơi tích hợp code mới nhất, tương ứng môi trường Dev.
  - `staging`: Code đã ổn định ở Dev, merge lên đây để kiểm thử gần giống thật, tương ứng môi trường Staging.
  - `main` (hoặc `production`): Code đã qua Staging, merge lên đây khi sẵn sàng release, tương ứng môi trường Production.
- **Quy tắc luồng code:** Feature mới → merge vào `dev` → sau khi kiểm tra ổn ở Dev thì merge lên `staging` → sau khi kiểm tra ổn ở Staging thì merge lên `main`. Không merge thẳng từ `dev` lên `main`, bỏ qua `staging`.

- **Ý nghĩa:** Mỗi môi trường phản ánh đúng một trạng thái "chín" khác nhau của code, giảm rủi ro đưa thẳng code chưa kiểm thử kỹ lên Production.

2.3. Jenkins Multibranch Pipeline

- **Multibranch Pipeline là gì:** Một loại Job trong Jenkins tự động phát hiện các nhánh có trong repository, mỗi nhánh tự tạo một pipeline riêng dựa trên `Jenkinsfile` có sẵn trong nhánh đó.
- **Cách dùng trong bài này:** Dùng chung một `Jenkinsfile` cho cả 3 nhánh, nhưng bên trong pipeline rẽ nhánh logic theo biến `env.BRANCH_NAME` (Jenkins tự cung cấp) để quyết định: build tag như thế nào, đọc file cấu hình nào, và gọi webhook sang máy CD nào.
- **Ví dụ rẽ nhánh cấu hình (minh họa ý tưởng, không cần chép nguyên văn):**

```
stage('Build Image') {
    steps {
        script {
            def envName = env.BRANCH_NAME == 'main' ? 'prod' :
                          env.BRANCH_NAME == 'staging' ? 'staging' : 'dev'
            sh "docker build -t myapp:${envName}-${env.BUILD_NUMBER} ."
        }
    }
}
```

- Mỗi lần build vẫn giữ nguyên yêu cầu ở Buổi 10: bắt buộc chạy `stage('Test')` trước, fail thì dừng pipeline, không build/push image.

2.4. Tách biệt Env/Config/Secret theo từng môi trường — nguyên tắc bắt buộc: không nằm trong code

- **Nguyên tắc quan trọng nhất của buổi này:** File cấu hình/secret thật của Staging và Production (connection string thật, JWT secret thật, API key thật...) **tuyệt đối không được commit lên Git repository** — vì đây là một nguồn code dùng chung cho nhiều môi trường, để lộ secret thật trong đó coi như lộ cho tất cả mọi người có quyền đọc repo.
- **Vậy các file này nằm ở đâu?** Phải **có sẵn từ trước, ngay tại máy đang cấu hình để build/deploy môi trường đó** — nghĩa là:
  - Trên máy đóng vai trò Staging: đã có sẵn file `appsettings.Staging.json` (hoặc `.env.staging`) chứa secret thật của Staging, đặt thủ công một lần bởi người vận hành, không đi qua Git.
  - Trên máy đóng vai trò Production: tương tự, có sẵn file secret riêng của Production.
  - Repo chỉ nên chứa **file mẫu** (`appsettings.Staging.json.example`, `.env.example`) liệt kê tên các biến cần có, không chứa giá trị thật, và các file thật phải nằm trong `.gitignore`.
- **Cách đưa các file này vào container lúc chạy:** Bind mount file cấu hình thật từ máy host (Staging/Production) vào container lúc `docker run`/`docker compose up` (ví dụ `-v /path/to/appsettings.Staging.json:/app/appsettings.Production.json`), thay vì để file này nằm trong image được build từ Jenkins — đảm bảo cùng một image build ra có thể chạy đúng ở bất kỳ môi trường nào, chỉ khác file mount vào lúc chạy.
- **Minh họa bằng JWT (đăng nhập):** Ứng dụng Web API có tính năng đăng nhập trả về JWT, mỗi môi trường dùng một `JWT_SECRET_KEY` khác nhau (đọc từ file cấu hình riêng của máy đó). Vì secret khác

nhau, một token được cấp ở Dev sẽ **không** verify được ở Staging/Production — đây là cách trực quan nhất để chứng minh trước lớp rằng secret thật sự tách biệt theo môi trường chứ không chỉ nói suông.

2.5. Rollback — Quay lại phiên bản trước khi gặp sự cố

- **Tiền đề đã có từ Buổi 10:** Mỗi lần build đều gắn tag rõ ràng (`${BUILD_NUMBER}` hoặc commit SHA, kèm tên môi trường như `prod-42`), nên registry luôn lưu lại đầy đủ lịch sử các version image đã từng deploy.
- **Cách rollback:** Tạo thêm một Jenkins Job dạng **Parameterized Build** (hoặc thêm parameter cho job hiện tại), cho phép người vận hành nhập/chọn một tag cũ, sau đó job chỉ chạy lại đúng `stage('Trigger Deploy')` với tag đó — gọi webhook sang máy CD tương ứng để máy đó `docker pull` đúng tag cũ rồi khởi động lại container, **không build lại từ code**.
- **Vì sao không build lại:** Rollback cần nhanh và chắc chắn dùng đúng bản đã từng được kiểm thử (test đã pass) trước đó, không nên build lại từ source (có thể source đã bị sửa tiếp sau đó).
- **Giới hạn cần biết khi demo:** Rollback image/code không tự động rollback database. Nếu migration ở version mới đã thay đổi schema không tương thích ngược, rollback code có thể khiến ứng dụng lỗi vì DB đã ở schema mới — ở mức bài học, chỉ cần giải thích rõ giới hạn này bằng lời, không bắt buộc phải rollback database thật.

2.6. Mô phỏng nhiều Server bằng máy của các thành viên trong nhóm

- Không cần thuê server thật: coi máy tính của từng thành viên là một "server" riêng biệt cho một môi trường, ví dụ máy TV1 = Dev server, máy TV2 = Staging server, máy TV3 = Production server. Máy chạy Jenkins (CI) có thể trùng với máy Dev hoặc là một máy riêng.
- Các máy kết nối với nhau qua **cùng một mạng LAN** (wifi lớp học/nhà trọ), gọi HTTP webhook qua địa chỉ IP LAN của từng máy — đúng kỹ thuật đã dùng ở Buổi 10, chỉ khác là giờ có 3 máy CD thay vì 1.
- Mỗi máy đã tự cấu hình sẵn file env/secret riêng của môi trường mình đóng vai (theo mục 2.4) từ trước khi buổi demo diễn ra.

3. Chuẩn bị thuyết trình và demo

- **Chuẩn bị bắt buộc:** Có sẵn một project Web API hoàn chỉnh (có tính năng đăng nhập JWT, có database và migration — kế thừa từ Buổi 10), với repo Private đã có 3 nhánh `dev`/`staging`/`main` và Jenkinsfile dùng Multibranch Pipeline.
- **Demo — thực hiện trên nhiều máy đóng vai trò các môi trường khác nhau, kết nối cùng một mạng LAN:**
    1. Giới thiệu nhanh cấu trúc 3 nhánh và chỉ ra trên từng máy Dev/Staging/Production đã có sẵn file cấu hình/secret riêng (mở file cho thấy — chứng minh các giá trị này **không** có trong Git bằng `git log`/`git show` chỉ thấy file `.example`, không thấy giá trị thật).
    2. **Bắt buộc — luồng code mới đến sản phẩm:** Thêm một thay đổi code nhỏ, commit và push lên nhánh `dev`. Bấm build trên Jenkins: pipeline chạy Test → Build → Push → Trigger Deploy sang máy Dev, kiểm tra ứng dụng trên máy Dev đã cập nhật.
    3. Merge nhánh `dev` lên `staging`, build lại: pipeline build ra tag khác (`staging-...`), deploy sang máy Staging — dùng file cấu hình/secret khác máy Dev.
    4. **Bắt buộc — chứng minh JWT secret tách biệt theo môi trường:** Đăng nhập lấy token ở máy Dev, đem token đó gọi thử một API cần xác thực trên máy Staging — chứng minh bị từ chối (401) do khác `JWT_SECRET_KEY`.
    5. Merge `staging` lên `main`, build và deploy sang máy Production tương tự.

6. **Bắt buộc — demo Rollback:** Giả lập một lỗi ở phiên bản mới nhất đang chạy trên Staging hoặc Production (ví dụ cố tình đưa một bug vào), sau đó dùng Jenkins Job rollback để chọn lại tag phiên bản trước đó, deploy lại và kiểm tra ứng dụng hoạt động đúng như trước khi có lỗi — **không build lại từ source**.

- **Thời gian:** Phân bổ thời gian hợp lý cho lý thuyết và demo, ưu tiên bước 2 (chứng minh code mới → sản phẩm mới xuyên suốt pipeline) và bước 6 (rollback).

4. Phân công công việc và ước lượng thời gian (Gợi ý)

Task	Người thực hiện (Gợi ý)	Ước lượng (Point)
Ôn lại pipeline Buổi 10 và lên kế hoạch mở rộng 3 môi trường	Nhóm trưởng	2
Nghiên cứu chiến lược nhánh Git (dev/staging/main)	TV1	2
Nghiên cứu Jenkins Multibranch Pipeline và rẽ nhánh theo `BRANCH_NAME`	TV1	3
Nghiên cứu tách Env/Config/Secret theo môi trường, nguyên tắc không commit secret	TV2	3
Cấu hình tính năng đăng nhập JWT với secret khác nhau theo môi trường	TV2	3
Nghiên cứu cơ chế Rollback (Parameterized Build) trong Jenkins	TV3	3
Chuẩn bị nội dung slide thuyết trình	Nhóm trưởng	2
Cập nhật Jenkinsfile thành Multibranch, thêm logic theo môi trường	TV1, TV2	5
Cấu hình 3 máy (Dev/Staging/Production) với file secret riêng, webhook listener	Cả nhóm	5
Cấu hình Job/Pipeline Rollback	TV3	3
Tổng hợp, kiểm tra và chạy thử demo trên nhiều máy	Cả nhóm	3
Luyện tập thuyết trình và demo	Cả nhóm	1
Tổng cộng		**35**

5. Tài liệu tham khảo

- **Tài liệu chính thức:** Jenkins Multibranch Pipeline Documentation, Jenkins Parameterized Build Documentation.
- **Bài viết:** Các bài viết về Git branching strategy (Git Flow, trunk-based development đơn giản hóa), best practices quản lý secret theo môi trường, JWT (JSON Web Token) và cách ký/verify bằng secret key.
- **Video:** Các video hướng dẫn Jenkins Multibranch Pipeline, quản lý config/secret nhiều môi trường, và rollback deployment.

6. Ghi chú quan trọng

- **Không commit secret thật, dù chỉ một lần:** Một khi giá trị nhạy cảm đã vào lịch sử Git, coi như đã lộ vĩnh viễn (dù xóa ở commit sau vẫn còn trong lịch sử) — luôn kiểm tra kỹ trước khi commit, dùng `.gitignore` ngay từ đầu.
- **Không dùng cùng một secret cho nhiều môi trường:** Đây là lỗi phổ biến khiến việc tách môi trường trở nên vô nghĩa — JWT secret, connection string của Production phải khác hẳn Dev/Staging.
- **Rollback là kỹ năng vận hành quan trọng:** Trong thực tế, khả năng quay lại phiên bản ổn định trước đó nhanh chóng quan trọng không kém khả năng deploy phiên bản mới — luôn giữ lại đủ lịch sử image/tag để có thể rollback khi cần.
- **Migration khi rollback cần cẩn trọng:** Hiểu rõ giới hạn là rollback code không tự rollback database, tránh gây lỗi dữ liệu khi thao tác trên môi trường thật.
- **Ba máy vẫn nên đồng bộ thời gian và múi giờ:** Tránh log/token có timestamp lệch nhau gây khó debug khi demo.

Chúc các bạn học tập hiệu quả và thành công!